Authentication

Introduction

Security lies at our heart, so we have taken steps to help you to process transactions more safely. To reduce the risk that someone other than you can use your integration, our platform uses security codes linked to your PSPID.

The so-called `API Key` and `API Secret` provide a unique signature to the traffic between your system and our platform. You can define these for each of your PSPIDs in the **Merchant Portal**. Any server-to-server request from your system to our platform (via your PSPID) needs to contain both the `API Key` and `API Secret`. Our platform tries to match the codes we receive from you with the ones configured in the Merchant Portal.

Our Server SDKs offer a simple solution for this mechanism, requiring only few lines of coding. If you wish to use your own software, **manual implementation** is also possible.

Target most up-to-date API base URL

> We encourage you to always target the most up-to-date API base URL when sending requests to our platform. Have a look at **our dedicated guide** for a full overview.
>
> To allow you a smooth transition, previous API base URLs remain available until further notice.

API key/secret configuration

> **Configuration in the Back Office**
>
> Are you using the Back Office?
>
> You can configure the `API key` / `secret` there as well. **Learn here how to do it**.
>
> The API key/secret you configure in the Back Office can also be managed in the Merchant Portal (and vice versa), as they share the same data set.

To configure the `API Key` / `Secret` pair in the Merchant Portal, follow these steps:

1. Login to the Merchant Portal. Go to **Developer > Payment API**
2. If you have not configured anything yet, the screen shows "No keys generated". To create both a `API Key` / `Secret` pair, click on “Add API Key”. The screen now shows both codes in the table in the “API Key ID” / “Secret API Key” line respectively

i

- **Each `API Key` / `Secret` pair expires one year after creation. Ensure timely renewal for uninterrupted transaction processing.**
- **Make sure to save the `API Secret` in your system immediately after its creation, as it will disappear from this screen after 60 seconds and never be visible anymore in the Merchant Portal. We strongly recommend you**
  - **Keep your API credentials somewhere safe and secure**
  - **Only disclose them to parties such as your developer, who require access for creating and managing your integrations**

You can look up the `API Key` and the following information about the `API Key` / `Secret` pair anytime via **Developer > Payment API**:

Expiration date	Status
The date when the `API Key` / `Secret` pair will expire (Default time of permanency is five years)	The current status of the `API Key` / `Secret` pair: • "Active": The currently active `API Key`. Usable until the date as defined in column "Expiration date" • "Expiring": Expires after the period as defined in column "Expiring in xh yym" • "Expired": Applicable to all `API Key` / `Secret` pairs you have ever configured for this PSPID

If you wish to use a new `API Key` / `Secret` pair to replace an existing one, follow these steps

1. Click on "Add API Key".

2. Click on "Confirm" in the dialogue box to proceed with the creation of a new `API Key` / `Secret` pair

> i **If you already have configured a `API Key` / `Secret` pair in your PSPID, creating a new pair will always revoke the existing one. Once created, the old pair will expire within four hours.**

Within this time, make sure to update any integrations using your old API/ Secret Key with the newly generated pair. This will prevent possible interruptions in your business activity

3 The screen now shows the new `API Key` / `Secret` pair at the bottom of the table in status "Active". The former active one now is in status "Expiring in xh yym"

> i **Make sure to save the `API Secret` in your system immediately after its creation, as it will disappear from this screen after 60 seconds and never be visible anymore in the Merchant Portal. We strongly recommend you**
>
> - **Keep your API credentials somewhere safe and secure**
> - **Disclose them to parties such as your developer, who require access for creating and managing your integrations**

Authentication with SDK

Once you have set up your `API Key/Secret` in the Merchant Portal, you can use them for requests you send to our platform. **Our Server SDKs** make it really easy to do so.

Have a look at this code sample that demonstrates how to add them in a standard Hosted Checkout Page request:

Java Copy

```
String yourAPIkey="yourAPIkey";
String yourAPIsecret="yourAPIsecret";
String yourPSPID="yourPSPID";

// Initiate fields for createHostedCheckoutRequest
CreateHostedCheckoutRequest createHostedCheckoutReq
        .withOrder(new Order()
            .withAmountOfMoney(new AmountOfMoney()
                .withAmount(100L)
                .withCurrencyCode("EUR")
            )
        );

// Initiate the communicator with our TEST/LIVE env
URI apiEndpoint = URI.create("https://payment.prepr

// Initialise the client with the apikey, apisecret
final Authenticator authenticator = new DefaultAuth
final MetaDataProvider metadataProvider = new MetaD

final DefaultConnection defaultConnection = new Def

ClientInterface client = Factory.createClient(apiEn

// Send the request to your PSPID on our platform a
CreateHostedCheckoutResponse createHostedCheckoutRe
```

```
.merchant(yourPSPID).hostedCheckout()
.createHostedCheckout(createHostedCheckoutR
```

> This authentication mechanism is only applicable to the Server API. The **Client API** employs a different mechanism.

Authentication without SDK

You can use our API without the SDKs by sending requests **to our endpoints** directly. This requires you to follow a different authentication mechanism.

Essentially, you need

- To hash with your `API Secret` over several HTTP headers (the `signature contents` or `signed-data`) you send along with your request
- Send this hash (the `signature`) together with your `API Key` to verify the authentication

works and our **GET**, **POST** and **DELETE** examples.

Understand authentication process

A GET/POST/DELETE request following this approach includes the following steps:

1. **Create a string-to-hash**, consisting of several HTTP headers
2. **Calculate the hash using** the algorithm HMAC-SHA256 with your API Secret
3. **Send the actual request** to our platform, including the headers, the hash and your API Key

1. Create string-to-hash

Compose the string-to-hash (the `signature contents` or `signed-data`) by following these rules:

Order of the headers

The string-to-hash consists of the following headers, listed in a specific order:

Copy

```
<HTTP method>
<Content-Type>
<Date>
<CanonicalizedHeaders>
<CanonicalizedResource>
```

Header Content Conventions

Each header is constructed as follows:

Copy

```
<lowercased header name>:<header value>;
```

Apply the following conventions for the content of the headers:

Headers	Conventions
`<HTTP method>`	Use either "GET," "POST," or "DELETE" (always in uppercase) depending on the HTTP method used.
`<Content-Type>`	Use the fixed value " `application/json; charset=utf-8` " for POST requests and use an empty string for GET or DELETE requests.
`<Date>`	Use the " `Date` " header in RFC1123 format.
`<CanonicalizedHeaders>`	Include custom headers specific to your application, such as `X-GCS-ClientMetaInfo`.
`<CanonicalizedResource>`	Include the target URI without the HTTP method, and include all decoded query parameters.

Unwrap Header Values

If a header value is wrapped over multiple lines, unwrap each line as described here. Unwrap by replacing a new line or multiple spaces by a single white space.

Have a look at this example:

Copy

```
A very long line<CR><LF>
<SPACE><SPACE><SPACE><SPACE>that does not fit on a single l
```

becomes

Copy

```
A very long line that does not fit on a single line
```

Trim White Spaces

Trim all whitespaces which exist at the start and end of the value.

Have a look at this example:

Copy

```
<space><space><space>Some text<space><space><space>
```

becomes

Copy

```
Some text
```

> Ensure that each `<item>` is followed by a new line (line feed), including the last item.

Have a look at this code sample for a **CreateHostedCheckout** request:

Java Copy

```java
public String createDateTime() {
    Calendar calendar = Calendar.getInstance();
    SimpleDateFormat dateFormat = new SimpleDateFor
    dateFormat.setTimeZone(TimeZone.getTimeZone("GM
    return dateFormat.format(calendar.getTime());
}

public String createStringToHash(String dateTime) {
    String stringToHash;

    String pspid = "YourPSPID";
    String requestMethod = "POST";
    String contentType = ContentType.APPLICATION_JS
    String uriResource = "/hostedcheckouts";
```

```
    String endpointURL = "/v2/" + pspid + uriResour

    stringToHash = requestMethod + "\n" + contentTy

    return stringToHash;
}
```

> Although the same rules apply for either HTTP method, you need to take the differences for GET/DELETE requests into account when creating the string-to-hash (I.e. missing content type, addition of query parameters to the request URI).
>
> Have a look at the **GET example** or the **DELETE example** in our dedicated chapter to know exactly what you need to do

2. Calculate hash

Once you have created the string-to-hash (the `signature contents` or `signed-data`), you need to hash it via the MAC algorithm HMAC-SHA256 with your `API Secret`.
The result of the hashing is the `signature` you need to provide in the authorisation header of your request.

Have a look at this code sample:

Java Copy

```java
public static String createHash(String stringToHash,
    // Convert stringToHash + key into byte array
    String algorithm = "HmacSHA256";
    byte[] keyAsBytes = yourApiSecret.getBytes("UTF-
    byte[] stringToHashAsBytes = stringToHash.getBy

    SecretKeySpec secretKeySpec = new SecretKeySpec
    Mac mac = Mac.getInstance(algorithm);
    mac.init(secretKeySpec);
    byte[] hash = mac.doFinal(stringToHashAsBytes);
    return Base64.encodeBase64String(hash);
}
```

3. Send request

Finally, you can send your request, including

- The headers you used for creating the string-to-hash (the `signature contents` or `signed-data`)
- The base64-encoded hash result (the “signature”) and your API Key (which our platform uses to identify your PSPID on our account) in the HTTP header 'Authorization' according to this formula:

Copy

```
Authorization = "GCS v1HMAC:" + API Key + ":" + signatu
```

- The JSON-encoded request body for POST requests (GET/DELETE do not require a request body)
  Have a look at this code sample:

Java Copy

```java
public static String sendRequest(String apiKey, Str
    CloseableHttpClient httpClient = HttpClients.cr
    String authorizationHeaderValue = "GCS v1HMAC:"

    String url = "https://payment.preprod.direct.wo

    if ("POST".equals(requestMethod)) {

        HttpPost httpPost = new HttpPost(url);

        // Build and send POST request
        String requestBodyAsString = "{"
                + "'order':{"
                + "    'amountOfMoney':{"
                + "        'currencyCode':'EUR',"
                + "                'amount':100"
                + "    }"
                + "}"
                + "} ";

        HttpEntity httpEntity = new StringEntity(re
        addHeaders(httpPost, dateTime, contentType,
        httpPost.setEntity(httpEntity);

        // Execute
        CloseableHttpResponse httpResponse = httpCl

        // Parse JSON response
        return parseResponse(httpResponse);

    } else if ("GET".equals(requestMethod)) {
        HttpGet httpGet = new HttpGet(url);
```

```
            addHeaders(httpGet, dateTime, contentType, a

            CloseableHttpResponse httpResponse = httpCl
            return parseResponse(httpResponse);
        } else if ("DELETE".equals(requestMethod)) {
            HttpDelete httpDelete = new HttpDelete(url),
            addHeaders(httpDelete, dateTime, contentTyp

            CloseableHttpResponse httpResponse = httpCl
            return parseResponse(httpResponse);
        } else {
            return "Something went wrong";
        }
    }

    private static void addHeaders(HttpUriRequest httpU
        httpUriRequest.addHeader(AUTHORIZATION, authori
        httpUriRequest.addHeader(DATE, dateTime);
        httpUriRequest.addHeader(CONTENT_TYPE, contentT
    }

    private static String parseResponse(CloseableHttpRe
        String responseString = EntityUtils.toString(ht
        JSONObject response = new JSONObject(responseSt
        return response.toString();
    }
```

Understand string-to-hash examples

As the general requirements for a GET/POST/DELETE request differ, the construction of the string-to-hash does as well. Have a look at the following examples to learn how to implement them in your system:

1. POST request

The string-to-hash (the `signature contents` or `signed-data`) for a **CreateHostedCheckout** request:

Copy

```
POST
application/json; charset=utf-8
Wed, 02 Mar 2022 11:15:51 GMT
/v2/yourPSPID/hostedcheckouts
```

> Mind that there is a new line (line feed) after the last header (URI resource) which you also need to consider when hashing the string

2. GET request

The string-to-hash (the `signature contents` or `signed-data`) for a **GetHostedCheckout** request:

Copy

```
GET

Wed, 02 Mar 2022 11:15:51 GMT
/v2/yourPSPID/hostedcheckouts/yourHostedCheckoutID
```

> Mind that

- There is an empty string for `Content-Type` header, as GET request do not have a request body
- There is a new line (line feed) after the last header (URI resource)

which you also need to consider when hashing the string

3. DELETE request

The string-to-hash (the `signature contents` or `signed-data`) for a **DeleteToken** request:

Copy

```
DELETE

Wed, 02 Mar 2022 11:15:51 GMT
/v2/yourPSPID/tokens/yourTokenID
```

Mind that

- There is an empty string for `Content-Type` header, as DELETE request do not have a request body
- There is a new line (line feed) after the last header (URI resource)

which you also need to consider when hashing the string

Was this page helpful? Yes No

Help & Info

Login/Password Reset

Contact us

Release Notes

About

API Info

API Reference

API Status

API Versioning

Developer Tools

Introduction API key/secret configuration Authentication wi

API Explorer

Template Builder

© 2026 | Worldline | Terms of use | Privacy | Cookie notice